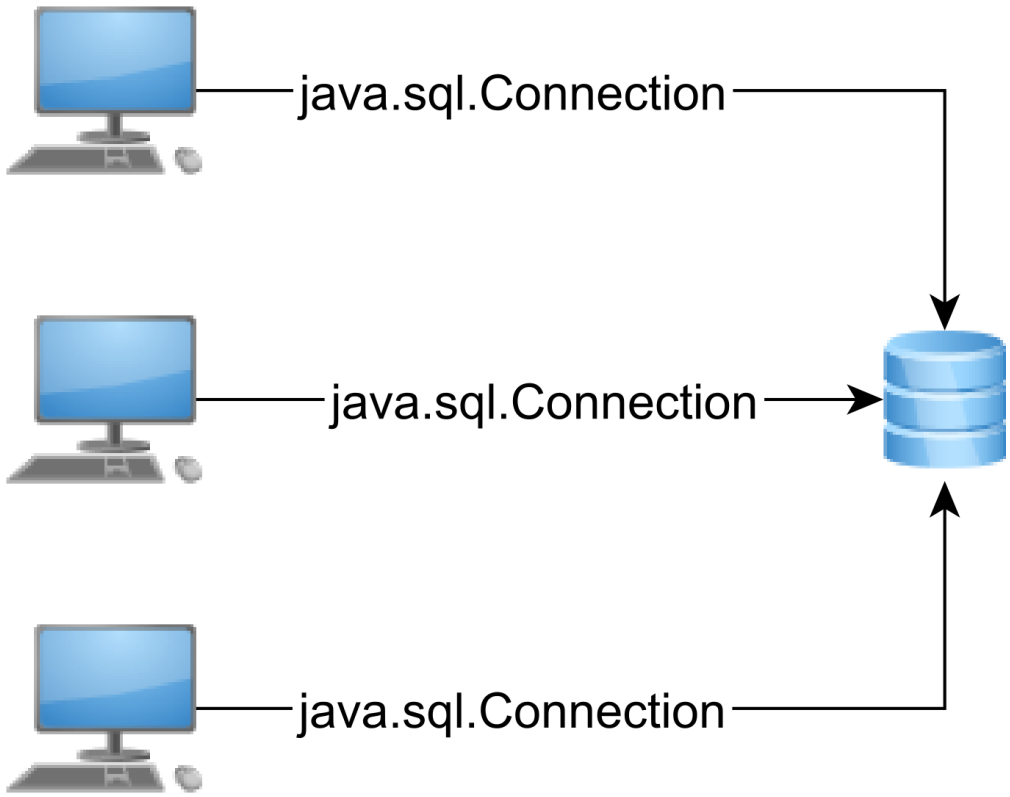

Figure 2.3: Two-tier architecture

In a two-tier architecture, the application is run by a single user, and each instance uses a dedicated database connection. The more users, the more database connections are required, and based on the underlying resources (hardware, operating system or licensing restrictions), each database server can offer a limited number of connections.

Oracle mainframe legacy

Oracle had gained its popularity in the era of mainframe computers when each client got a dedicated database connection.

Oracle assigns a distinct *schema* for each individual *user*, as opposed to other database systems where a schema is shared by multiple user accounts.

In PL/SQL, the *Packaged public variables* scope is bound to a *session*, instead of to the currently running transaction. The application developer must be extra cautious to unbind these variables properly since connections are often reused and old values might leak into newer transactions.

2.2 DataSource

In 1999, J2EE was launched along with JDBC 2.0 and an initial draft of JTA (Java Transaction API)[1], marking the beginning of Enterprise Java. Enterprise applications use a three-tier architecture, where the middle tier acts as a bridge between user requests and various data sources (e.g. relational databases, messaging queues).

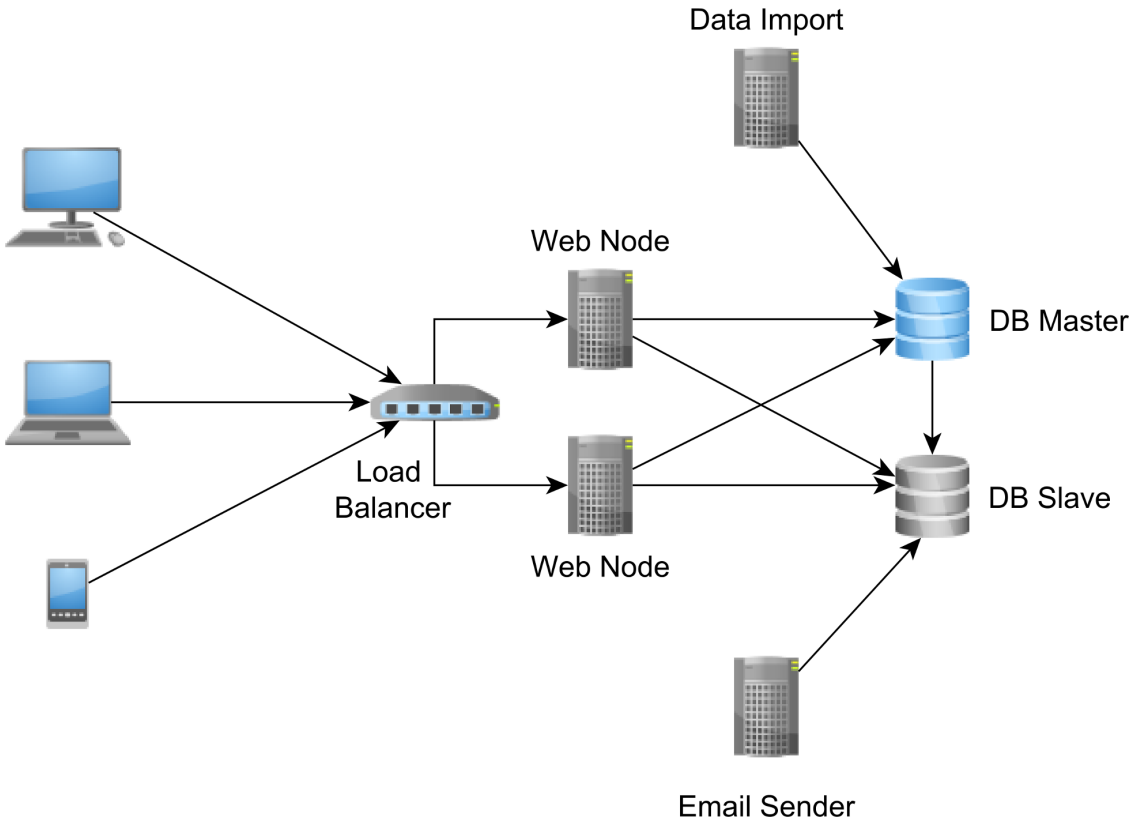

Figure 2.4: Three-tier architecture

Having an intermediate layer between the client and the database server has numerous advantages.

In a typical enterprise application, the user request throughput is greater than the available database connection capacity. As long as the connection acquisition time is tolerable (from the end-user perspective), the user request can wait for a database connection to become available. The middle layer acts as a database connection buffer that can mitigate user request traffic spikes by increasing request response time, without depleting database connections or discarding incoming traffic.

Because the intermediate layer manages database connections, the application server can also monitor connection usage and provide statistics to the operations team.

For this reason, instead of serving physical database connections, the application server

[1] https://jcp.org/en/jsr/detail?id=907

provides only logical connections (proxies or handles), so it can intercept and register how the client API interacts with the connection object.

A three-tier architecture can accommodate multiple data sources or messaging queue implementations. To span a single transaction over multiple sources of data, a distributed transaction manager becomes mandatory. In a JTA environment, the transaction manager must be aware of all logical connections the client has acquired as it has to commit or roll them back according to the global transaction outcome. By providing logical connections, the application server can decorate the database connection handles with JTA transaction semantics.

If the `DriverManager` is a physical connection factory, the `javax.sql.DataSource` interface is a logical connection provider:

```
Connection getConnection() throws SQLException;
Connection getConnection(String username, String password) throws SQLException;
```

The simplest `javax.sql.DataSource` implementation could delegate connection acquisition requests to the underlying `DriverManager`, and the connection request workflow would look like this:

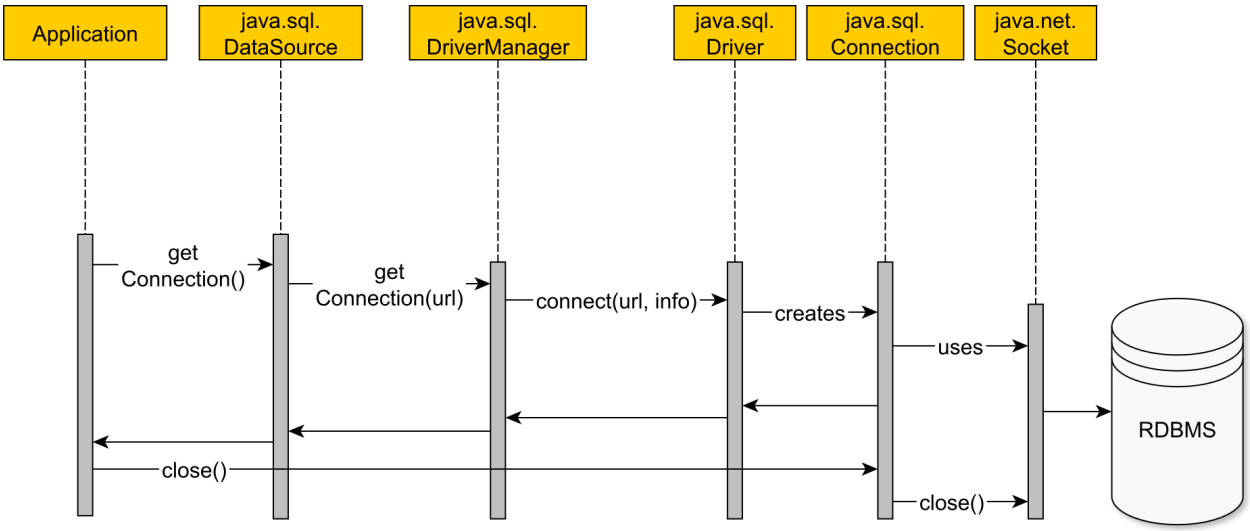

Figure 2.5: DataSource without connection pooling

1. The application data layer asks the `DataSource` for a database connection.
2. The `DataSource` uses the underlying driver to open a physical connection.
3. A physical connection is created, and a TCP socket is opened.
4. The `DataSource` under test does not wrap the physical connection, and it simply lends it to the application layer.
5. The application executes statements using the acquired database connection.
6. When the connection is no longer needed, the application closes the physical connection along with the underlying TCP socket.

Opening and closing database connections is a very expensive operation, so reusing them has the following advantages:

- It avoids both the database and the driver overhead for establishing a TCP connection.
- It prevents destroying the temporary memory buffers associated with each database connection.
- It reduces client-side JVM object garbage.

To visualize the cumulated overhead of establishing and closing database connections, the following test compares the total time it takes to open and close 1000 database connections of four different RDBMS against HikariCP[2] (one of the fastest stand-alone connection pooling solutions in the Java ecosystem).

Table 2.1: Database connection establishing overhead vs. connection pooling

Metric	Time (ms) DB_A	Time (ms) DB_B	Time (ms) DB_C	Time (ms) DB_D	Time (ms) HikariCP
min	11.174	5.441	24.468	0.860	0.001230
max	129.400	26.110	74.634	74.313	1.014051
mean	13.829	6.477	28.910	1.590	0.003458
p99	20.432	9.944	54.952	3.022	0.010263

When using a connection pooling solution, the connection acquisition time is between two and four orders of magnitude smaller. By reducing the connection acquisition interval, the overall transaction response time gets shorter too. All in all, in an enterprise application, reusing connections is a much better choice than always establishing them on a transaction basis.

Oracle XE connection handling limitation

While the Enterprise Edition does not entail any limitations, the Oracle 11g Express Edition throws the following exception when running very short transactions without using a connection pooling solution:

> ORA-12516, TNS:listener could not find available handler with matching protocol stack

A connection pooling solution can prevent these intermittent connection establishment failures and reduce the connection acquisition time as well.

[2] http://brettwooldridge.github.io/HikariCP/

2.2.1 Why is pooling so much faster?

To understand why the connection pooling solution performs so much better, it is important to figure out the connection pooling mechanism:

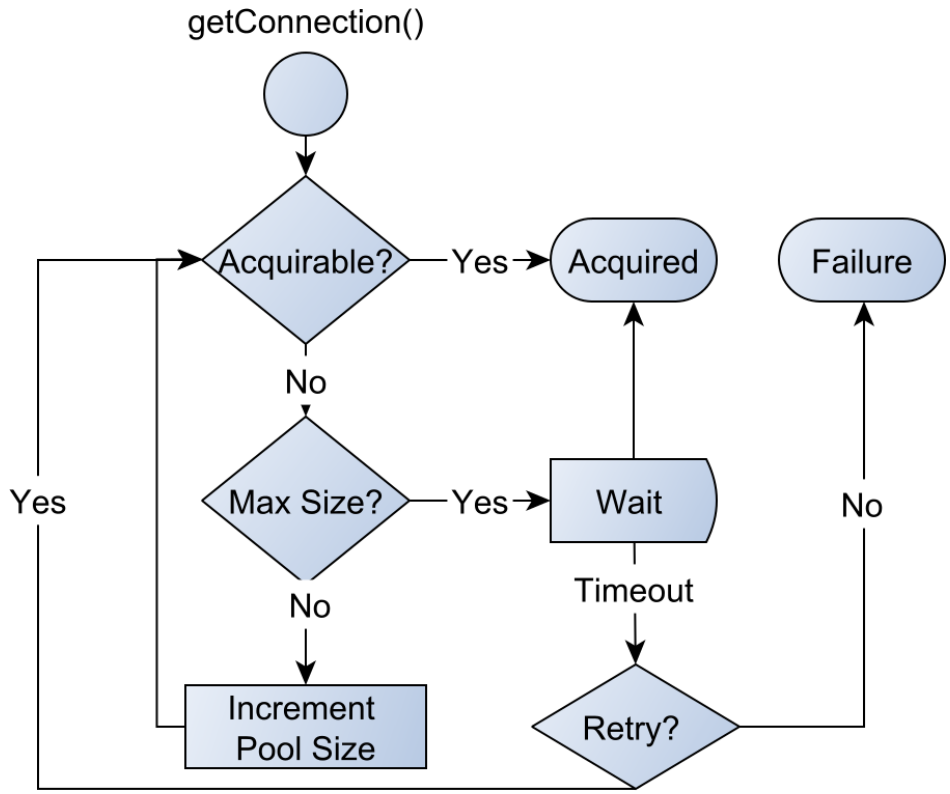

Figure 2.6: Connection acquisition request flow

1. When a connection is being requested, the pool looks for unallocated connections.
2. If the pool finds a free one, it will be handled to the client.
3. If there is no free connection, the pool will try to grow to its maximum allowed size.
4. If the pool already reached its maximum size, it will retry several times before giving up with a connection acquisition failure exception.
5. When the client closes the logical connection, the connection is released and returns to the pool without closing the underlying physical connection.

Most connection pooling solutions expose a `DataSource` implementation that either wraps an actual database-specific `DataSource` or the underlying `DriverManager` utility.